

OSSP Skill Week 10

Task 1: Signal Handlers

Aim

To register handlers for SIGINT and SIGTERM, configure their actions, and verify that the process stays stable when interrupts arrive.

Program

```
#include <stdio.h>
#include <signal.h>
#include <string.h>
#include <unistd.h>

/* Handlers use only async-signal-safe functions */
static void signal_handler(int sig)
{
    if (sig == SIGINT) {
        const char msg[] = "\nSIGINT received. Handling interrupt...\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    } else if (sig == SIGTERM) {
        const char msg[] = "\nSIGTERM received. Handling termination...\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
        _exit(0);
    }
}

int main(void)
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;

    if (sigaction(SIGINT, &sa, NULL) < 0 || sigaction(SIGTERM, &sa, NULL) < 0) {
        perror("sigaction");
        return 1;
    }

    printf("Signal handler program started.\n");
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C to send SIGINT.\n");

    while (1) {
        printf("Process is running...\n");
        sleep(2);
    }

    return 0;
}
```

Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % gcc signal_handler.c -o signal_handler
hrushika@Hrushikas-MacBook-Air pjsdj % ./signal_handler
Signal handler program started.
PID: 75014
Press Ctrl+C to send SIGINT.
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
Process is running...
^C
SIGINT received. Handling interrupt...
Process is running...
Process is running...
^C
SIGINT received. Handling interrupt...
Process is running...
Process is running...
```

Observation

When Ctrl+C generated SIGINT, the registered handler ran instead of the default action, and the process continued printing its status. SIGTERM ran the handler, which ended the process cleanly.

Result

Signal handlers were registered successfully and interrupt events were handle

Task 2: SIGINT Handling

Aim

To show how a shell process stays active while it forwards SIGINT to its foreground child process.

Program

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <sys/wait.h>

static volatile pid_t child_pid = -1;

static void handle_sigint(int sig)
{
    (void)sig;
    if (child_pid > 0) {
        const char msg[] = "\nShell received SIGINT. Forwarding to foreground job...\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
        kill(child_pid, SIGINT);
    } else {
        const char msg[] = "\nShell protected from SIGINT.\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    }
}

int main(void)
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    if (sigaction(SIGINT, &sa, NULL) < 0) {
        perror("sigaction");
        return 1;
    }

    printf("Shell started. PID: %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Foreground job: default SIGINT action, own process group */
        signal(SIGINT, SIG_DFL);
        setpgid(0, 0);

        printf("Foreground job started. PID: %d\n", getpid());
        while (1) {
            printf("Foreground job running...\n");
            fflush(stdout);
            sleep(2);
        }
    }

    setpgid(pid, pid);
    child_pid = pid;

    printf("Foreground job running with PID %d\n", pid);
    printf("Press Ctrl+C to send SIGINT.\n");
    fflush(stdout);

    /* Retry if waitpid() is interrupted by the signal handler */
    int status;
    while (waitpid(pid, &status, 0) < 0) {
        if (errno != EINTR) {
            perror("waitpid");
            return 1;
        }
    }

    child_pid = -1;

    if (WIFSIGNALED(status))
        printf("Foreground job terminated by signal %d.\n", WTERMSIG(status));
    printf("Job state updated.\n");
    printf("Shell is still running.\n");

    return 0;
}
```

Output

```
hrushika@Hrushikas-MacBook-Air pjsdj % gcc sigint.c -o sigint
hrushika@Hrushikas-MacBook-Air pjsdj % ./sigint
Shell started. PID: 75666
Foreground job running with PID 75667
Press Ctrl+C to send SIGINT.
Foreground job started. PID: 75667
Foreground job running...
Foreground job running...
Foreground job running...
^C
Shell received SIGINT. Forwarding to foreground job...
Foreground job terminated by signal 2.
Job state updated.
Shell is still running.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The shell's handler caught SIGINT and forwarded it to the foreground child with `kill()`. The child terminated with signal 2 (SIGINT), while the shell process survived and updated the job state.

Result

SIGINT capture, signal forwarding and foreground job termination were demonstrated successfully.

Task 3: SIGTSTP and Job Suspension

Aim

To capture SIGTSTP, suspend a foreground process, preserve its job state and show that the process can be resumed later.

Program

```
#include <stdio.h>
#include <signal.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

static volatile pid_t child_pid = -1;
static volatile sig_atomic_t suspended = 0;

static void handle_sigstsp(int sig)
{
    (void)sig;
    const char msg[] = "\nSIGTSTP received. Suspending foreground job...\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);

    if (child_pid > 0)
        kill(child_pid, SIGSTOP);
    suspended = 1;
}

int main(void)
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = handle_sigstsp;
    sigemptyset(&sa.sa_mask);
    if (sigaction(SIGTSTP, &sa, NULL) < 0) {
        perror("sigaction");
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        signal(SIGTSTP, SIG_DFL);
        setpgid(0, 0);
        while (1) {
            printf("Foreground job running...\n");
            fflush(stdout);
            sleep(2);
        }
    }

    setpgid(pid, pid);
    child_pid = pid;
```

```
    setpgid(pid, pid);
    child_pid = pid;

    printf("Foreground job started. PID: %d\n", pid);
    printf("Press Ctrl+Z to suspend the job.\n");
    fflush(stdout);

    sleep(3); /* Returns early if Ctrl+Z is pressed */

    if (!suspended) {
        printf("Sending SIGTSTP to foreground job...\n");
        fflush(stdout);
        raise(SIGTSTP);
    }

    int status;
    if (waitpid(pid, &status, WUNTRACED) > 0 && WIFSTOPPED(status))
        printf("Job state: Stopped (state preserved).\n");

    sleep(2);

    printf("Resuming stopped job using SIGCONT...\n");
    fflush(stdout);
    kill(pid, SIGCONT);

    sleep(3);

    kill(pid, SIGTERM);
    waitpid(pid, NULL, 0);
    child_pid = -1;

    printf("Job completed and removed from job table.\n");
    return 0;
```

Output

```
Job state updated.
Shell is still running.
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc sigtstp.c -o sigtstp
⦿ hrushika@Hrushikas-MacBook-Air pjsdj % ./sigstp
zsh: no such file or directory: ./sigstp
⦿ hrushika@Hrushikas-MacBook-Air pjsdj % ./sigtstp
Foreground job started. PID: 75917
Press Ctrl+Z to suspend the job.
Foreground job running...
Foreground job running...
Sending SIGTSTP to foreground job...

SIGTSTP received. Suspending foreground job...
Job state: Stopped (state preserved).
Resuming stopped job using SIGCONT...
Foreground job running...
^C
⦿ hrushika@Hrushikas-MacBook-Air pjsdj % Foreground job running...
Foreground job running...
Foreground job running...
❖ hrushika@Hrushikas-MacBook-Air pjsdj % Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
```

Observation

SIGTSTP was caught by the handler, which suspended the foreground child with SIGSTOP. The child printed nothing while stopped, its state was preserved, and after SIGCONT it continued running until it was terminated and removed.

Result

Job suspension, state preservation and later recovery were demonstrated successfully.

Task 4: Process Groups

Aim

To create a process group, assign a process to it, retrieve the process group ID and demonstrate group-based signal delivery.

Program

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        /* Child becomes the leader of a new process group */
        if (setpgid(0, 0) < 0) {
            perror("setpgid (child)");
            exit(1);
        }

        printf("Child PID: %d\n", getpid());
        printf("Process Group ID: %d\n", getpgrp());
        fflush(stdout);

        sleep(10);
        printf("Child process exiting.\n");
        exit(0);
    }

    /* Set it from the parent too, avoiding a race with the child */
    setpgid(pid, pid);
    sleep(1);

    pid_t pgid = getpgid(pid);
    if (pgid < 0) {
        perror("getpgid");
        return 1;
    }

    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);
    printf("Child Process Group ID: %d\n", pgid);

    printf("Sending SIGTERM to process group...\n");
    if (kill(-pgid, SIGTERM) < 0) {
        perror("kill");
        return 1;
    }

    waitpid(pid, NULL, 0);
    printf("Process group execution completed.\n");

    return 0;
}
```

Output

```
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc process_group.c -o process_group
● hrushika@Hrushikas-MacBook-Air pjsdj % ./process_group
Child PID: 76755
Process Group ID: 76755
Parent PID: 76754
Child PID: 76755
Child Process Group ID: 76755
Sending SIGTERM to process group...
Process group execution completed.
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The child became the leader of a new process group, so its group ID equalled its PID. The parent read this ID with `getpgid()` and sent `SIGTERM` to the whole group using the negative group ID, which ended the child before its 10-second sleep finished.

Result

Process group creation, group identification and group-based signal delivery were implemented successfully.